

False Positives, Real Problems: Evaluating Static Analysis Tools

Falska larm, riktiga problem: En utvärdering av statiska kod-analysverktyg

Edvin Dyremark
Tobias Elfstrand

Supervisor : Animesh Thakur
Examiner : Mikael Asplund

External supervisors : Johan Larsson & Daniel Thorén

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

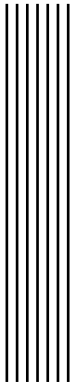
For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Abstract

Static analysis tools are widely used to detect vulnerabilities in C/C++ code, yet their effectiveness across realistic and synthetic datasets remains underexplored. This thesis systematically evaluates five prominent static analysis tools: Cppcheck, Flawfinder, Semgrep, Clang-Tidy, and Snyk Code. The evaluation was done using two contrasting datasets: the Juliet Test Suite, offering synthetic examples with clearly labeled vulnerabilities, and PrimeVul, providing real-world vulnerable functions paired with their corresponding patches. An evaluation system was implemented, including automated build support across diverse open-source projects, tool output normalization, and metric-based classification of results. Performance was assessed using many metrics, such as Precision, Recall, F1, pair-wise comparative measures, and the Matthews Correlation Coefficient.

The results reveal that all tools perform relatively poorly on the synthetic dataset and really poorly on the realistic dataset. This highlights the need for tool improvement and dataset improvements, as well as the importance of dataset diversity in benchmarking static analysis tools.

Keywords: Static Analysis, Static Analysis tools, Juliet Test Suite, PrimeVul, Cybersecurity

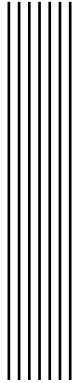


Acknowledgments

We would like to express our gratitude to all those who supported and contributed to the completion of this thesis. In particular, we would like to thank Johan Larsson and Daniel Thorén, our supervisors at Sectra Communications AB for the valuable discussions and providing access to the necessary resources throughout the project. We are also grateful to Sectra Communications AB for the opportunity to carry out this work within their organization.

We would also like to thank our examiner, Mikael Asplund, and our academic supervisor, Animesh Thakur, at the Department of Computer and Information Science (IDA) at Linköping University, for their valuable feedback, insightful comments, and support throughout the thesis process.

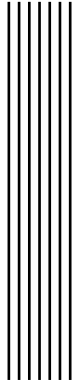
Finally, we extend our heartfelt thanks to our friends and fellow students who made our time at Linköping University both enjoyable and memorable.



Contents

Abstract	iii
Acknowledgments	iv
Contents	v
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	1
1.2 Aim	2
1.3 Research questions	2
1.4 Approach	2
1.5 Delimitations	3
1.6 Contributions	3
2 Background and Related Work	4
2.1 Background	4
2.2 Related work	9
3 Method	12
3.1 Datasets	12
3.2 Workflow	14
3.3 Extended Data Processing	18
4 Results	20
4.1 Performance on Juliet	20
4.2 Performance on PrimeVul	23
5 Discussion	27
5.1 Results	27
5.2 Method	33

5.3 The work in a wider context	35
6 Conclusion	36
6.1 Research questions	36
6.2 Future work	37
Bibliography	39
A Large-Language Models	42
B Union Data	45



List of Figures

2.1	Diagram of static analysis techniques.	5
3.1	Example of a vulnerable function from the Juliet Test Suite.	13
3.2	Example of a benign function from the Juliet Test Suite.	13
3.3	Example of a vulnerable function from PrimeVul.	14
3.4	Example of a benign function from PrimeVul.	14
3.5	Workflow diagram for evaluating SATs.	15
3.6	Workflow diagram for building compile database.	16
4.1	Classification distribution on Juliet Test Suite.	21
4.2	Classification distribution on MITRE top 25 subset.	21
4.3	SAT Classification distribution on PrimeVul.	24

List of Tables

2.1	Classification rules for fundamental metrics.	7
2.2	Pair-wise classification categories for functions in the PrimeVul dataset.	8
3.1	High-level statistics for the Juliet 1.3 C/C++ test suite.	12
3.2	Tools and their corresponding flags used in the analysis.	16
3.4	CWE categories for “Top 25” analysis on Juliet.	18
4.1	Classification results for SATs on the Juliet Test Suite.	20
4.2	Classification results for SATs MITRE top 25 CWEs.	21
4.3	Precision, Recall, F1-score and MCC on the Juliet Test Suite.	22
4.4	Intersection counts for TP and FP on the Juliet dataset.	22
4.5	Successful run and DNF rates for SATs on the PrimeVul dataset.	23
4.6	Percentage distribution of classification results for SATs and LLMs on the PrimeVul dataset.	23
4.7	Precision, Recall, F1-score, and MCC on the realistic dataset.	24
4.8	Pairwise successful run and Did Not Finish (DNF) rates for SATs on the PrimeVul dataset.	25
4.9	Pair-wise Correct Prediction, Vulnerable Prediction, Benign Prediction and Reversed Prediction on the realistic data.	25
4.10	Intersection counts for TP and FP on PrimeVul dataset.	26
5.1	Summation of TP and FP counts across singles, pairs, and triplets.	30
5.2	Precision and recall on the Juliet dataset compared with values reported by Gentsch.	32
5.3	Tool-specific limitations and remaining datapoints from real-world dataset.	34
A.1	Training parameters for LLM fine-tuning.	43
A.2	Percentage distribution of classification results for CodeBERT and T5.	43
A.3	Precision, Recall, F1-score, and MCC for T5 and CodeBERT on the realistic dataset.	44
A.4	Performance compared with PrimeVul study.	44
B.1	Union counts for TPs and FPs on the Juliet Dataset.	45
B.2	Union counts for TPs and FPs on the PrimeVul Dataset.	46



1 Introduction

This master's thesis investigates the performance of static analysis tools (SATs) based on different underlying technologies. The work was carried out at Sectra Communications in Linköping, Sweden. This chapter presents the problem addressed in the thesis, the motivation behind the study, and contributions to the field.

1.1 Motivation

Cybersecurity and secure development is gaining increased importance as cybercrime continues to rise. In 2024, losses in the U.S. alone exceeded \$16 billion, reflecting a 33% increase from the previous year [17]. One of the key challenges in modern software development is balancing the need for security with the constraints of limited time and resources. While dedicating additional effort to high-risk components can help reduce vulnerabilities, the growing demand for software makes it impractical to ensure complete security across all systems.

Manual code reviews, where no tool is used, is a common approach to enhance the security of software. However, this can be time-consuming and requires some level of expertise to be effective. Static analysis is another popular approach which requires less manual work. SATs checks code without execution, identifying bugs and potential vulnerabilities early in development. These tools are becoming increasingly popular in software development as security demands grow, with approximately 50% of open-source projects implementing some kind of automated analysis tool [1].

SATs use different methods to identify potential vulnerabilities [12]. Examining their strengths and weaknesses, it is possible to create more secure programs with less effort. Knowing when different tools and methods excel is crucial for using them effectively.

There are several approaches on how to compare and measure the performance of SATs, such as looking at performance on synthetic data [12], running the tool on open-source projects and afterwards labeling the results [20], or looking at one specific project [7].

This is particularly beneficial for tools based on abstract syntax trees (ASTs), which benefit from contextual information to function effectively. These comprehensive datasets can

provide the necessary code context, improving the accuracy and reliability of such tools in practical applications.

While several prior studies have evaluated SATs on either synthetic or real-world data, none have systematically examined tool performance across a wide range of open-source projects using a labeled dataset. One reason for this gap is that large, labeled datasets for vulnerability detection have only recently become available, largely driven by the growing popularity of machine learning. Machine learning models, particularly in the domains of code analysis and natural language processing, require extensive labeled data for training and evaluation. As a result, the demand for such datasets has led to large efforts to curate and share high-quality, annotated vulnerability data.

Understanding how these tools perform in realistic settings and how their performance can be attributed to different analysis technologies is essential. Evaluating effectiveness across a range of real-world projects enables a deeper assessment of generalizability and reliability, offering insight into the strengths and limitations of each tool in a wider context.

1.2 Aim

This thesis aims to address a gap in current research by performing a comprehensive evaluation of SATs focusing on C/C++ code, that are based on different technologies, with a focus on security. This comprehensive evaluation is conducted on two distinct datasets. The first consists of synthetic data, which allows for method validation through comparison with previous studies and enables tools to be tested on identical vulnerabilities presented at varying levels of difficulty. The second dataset comprises real-world historical data, providing insights into how the tools perform in realistic usage scenarios. A range of evaluation metrics is calculated and used to compare tool performance across the two datasets.

1.3 Research questions

This thesis answers the following research questions:

- 1.1. How do static analysis tools based on different technologies perform when applied to synthetic datasets?
- 1.2. How do static analysis tools based on different technologies perform when applied to real-world projects?

1.4 Approach

This study follows a three-stage approach to answer the research questions. Initially, the research phase identifies tools for evaluation by investigating popularity and relevant literature.

In the second stage, the tools are evaluated on two selected datasets: the Juliet Test Suite [2] and the PrimeVul dataset [8]. Tool outputs are standardized and documented to ensure consistent representation and reproducibility.

In the third and final stage, results from both datasets are compared using the predefined metrics, such as F1-score or Matthews Correlation Coefficient (MCC). This evaluation highlights the strengths and weaknesses of each tool, providing insights about their performance in different use cases.

1.5 Delimitations

This study focuses on SATs for C and C++ code. The following aspects are considered outside the scope of this project:

- Any form of dynamic analysis, including but not limited to runtime analysis, fuzz testing, and symbolic execution.
- SATs designed for languages other than C and C++.
- Performance benchmarking of analyzed SATs, as the study is centered on detection capabilities rather than execution efficiency.

1.6 Contributions

The contributions of this thesis are the following:

- A cleaned and updated dataset based on PrimeVul, with added line-numbers and unreachable projects removed.
- A detailed comparison between open-source, machine learning-based and commercial SATs when used on synthetic data versus real-world data.



2 Background and Related Work

This section presents the background essential for understanding the thesis and explores related work on SAT comparisons, and the role of machine learning in this field.

2.1 Background

We must first review the core concepts and techniques underlying static code analysis to understand how SATs perform in vulnerability detection. Static analysis is defined as "*The process of evaluating a system or component based on its form, structure, content, or documentation*"[16].

2.1.1 Static analysis Techniques

Static analysis techniques can be split into different categories where the hierarchy can be found in Figure 2.1.

Static analysis techniques for vulnerability detection can initially be grouped into two categories: *code similarity-based* and *pattern-based*. Code similarity-based tools are effective at identifying vulnerabilities caused by code cloning but tend to exhibit a high FN rate when vulnerabilities arise from other factors [21]. One advantage of these tools is that detecting a single instance of vulnerable code can potentially uncover similar issues in future programs. Pattern-based techniques are further divided into *rule-based* and *machine learning-based* approaches. Rule-based methods rely on a sufficient number of examples for rules to be defined, either manually by experts or through machine learning. In contrast, code similarity-based approaches are restricted to detecting vulnerabilities that are identical or nearly identical to known code clones, and are less effective when code has undergone modifications such as deletions, insertions, or reordering of statements [22].

Rule-based techniques use predefined rules to detect vulnerabilities. These rules are manually created by human experts, but have the disadvantage that there is often a high FP rate [4, 21].

Machine learning-based techniques are further divided into three parts, which include *vulnerability-prediction models* that use process metrics such as code churn, *anomaly-detection models* that look at the source code and find abnormal patterns such as strange API usage, and *pattern-recognition models* that learn vulnerable code snippets directly [21].

The third overall category is called Semantic Analysis, which approach reason about a programs behaviour rather than its surface syntax. They split naturally into two principal techniques: *abstract interpretation*, which over-approximates all paths using mathematical abstractions and *symbolic execution*, which explores concrete paths with symbolic inputs.

Abstract interpretation analyses every path at once by propagating abstract values, intervals, octagons, taint sets, etc, through the control-flow graph. Because the abstraction is conservative, the analysis terminates and scales to large projects, at the cost of possible FPs [25].

Symbolic execution replaces concrete values with symbols and generate path constraints that an SMT solver checks for feasibility. This allows them to uncover deep, data-dependent bugs such as buffer or integer overflows, that pattern matchers often miss [19]. Naturally, the number of paths to explore grows exponentially with program complexity, which is problematic for large projects like the ones examined in this thesis.

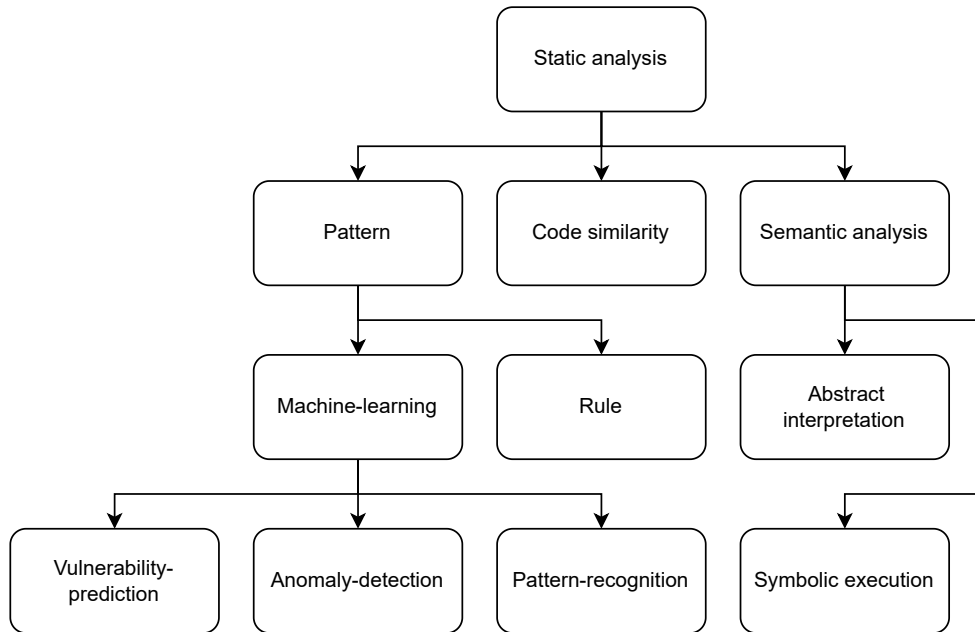


Figure 2.1: Diagram of static analysis techniques.

2.1.2 Static analysis tools

The study evaluates five widely-used C/C++ static-analysis tools (SATs) that rely on diverse techniques, including rule-based, pattern-based, and machine-learning-based approaches.

The tools were chosen based on three factors:

1. **Relevance and popularity:** tools that are widely adopted in C/C++ development.

2. **Prior research:** tools that are frequently occurring in academic literature.
3. **Underlying techniques:** a balanced mix of rule-based, AST, and AI- solutions.

Because reliable usage statistics are hard to find for these tools, relevance and popularity were decided from recent studies and user reports. Preference was given to tools that are actively maintained or released within the past decade, rather than those first launched more than 15 years ago, to better reflect current projects and use-cases.

Finally, tool selection was made to include multiple underlying technologies such as rule-based, AST and AI, which makes for a more interesting and informative comparison between different SAT technologies.

Cppcheck

Cppcheck is an open-source, rule-based analyzer focused on finding undefined behavior and potential security vulnerabilities in the code through a comprehensive list of checks as well as a bidirectional data flow analysis approach, allowing it to identify certain vulnerabilities that purely path-sensitive analyzers might miss^{1,2}.

Flawfinder

Flawfinder uses pattern-based matching, comparing code to a database of well-known problems, such as functions with buffer overflow risks, string problems, race conditions, potential shell dangers, or poor random number acquisitions. The benefit of using Flawfinder instead of grep on the source code is that Flawfinder will ignore comments and insides of strings as well as estimate a risk level depending on the parameters³.

Semgrep

Semgrep is a subscription-based SAT that uses rule-based matching. Semgrep has 53 default rules which were used for the study⁴. Semgrep rules are easily customized and expanded by the user⁵. There are rule sets created by the Semgrep community that can be downloaded and applied to scans⁶.

Clang-Tidy

Clang-Tidy is based on the clang static analyzer, which implements path-sensitive, inter-procedural analysis based on symbolic execution techniques⁷. It uses an AST-based analysis, enabling it to do cross translation unit analysis which requires the analyzed projects to be built and a compilation database to be generated^{8,9}. Clang-Tidy implements the same checks as clang static analyzer, meaning it is a Rule-based SAT. For this study, only the security-based checks were used¹⁰. Clang-Tidy requires a compilation database which is a file that specifies how each translation unit is compiled within the project.

¹<https://www.Cppcheck.com/what-is-static-code-analysis>

²https://cppcheck.sourceforge.io/devinfo/doxyoutput/group__Checks.html

³https://dwheeler.com/flipfinder/#how_work

⁴<https://semgrep.dev/p/c>

⁵<https://Semgrep.dev/docs/writing-rules/overview>

⁶<https://Semgrep.dev/explore>

⁷<https://clang.llvm.org/docs/ClangStaticAnalyzer.html>

⁸<https://clang.llvm.org/docs/analyzer/user-docs/CrossTranslationUnit.html>

⁹<https://clang.llvm.org/docs/JSONCompilationDatabase.html>

¹⁰<https://clang.llvm.org/extra/clang-tidy/>

Snyk Code

Snyk offers a suite of security-focused services. The SAT evaluated in this study is Snyk Code. For brevity, we refer to it simply as Snyk in the remainder of this work.

Snyk uses an AI-powered semantic analysis engine alongside a continuously updated, rule-based system with known Common Weakness Enumerations (CWE) and Open Web Application Security Project categories^{11,12}. It combines data-flow and control-flow analysis with predefined security rules. Snyk is an AI-driven engine that operates on a subscription-based model with limited transparency regarding its internal mechanisms. The proprietary nature of the tool means that the exact methodologies used for vulnerability detection, rule definitions, and AI training data remain undisclosed.

2.1.3 Metrics

This study adopts some of the standard classification metrics used in vulnerability-detection research to assess SAT performance. The subsections below define each metric formally and outline its relevance to static-analysis evaluation.

Fundamental metrics

The fundamental metrics track if something a tool flags is benign or vulnerable and correctly or incorrectly flagged. A summary of the fundamental classification rules is found in Table 2.1.

Table 2.1: Classification rules for fundamental metrics.

Classification	Explanation
True Positive (TP)	A security warning exists in a vulnerable function
False Positive (FP)	A security warning exists in a benign function
True Negative (TN)	No security warning exists, and the function is benign
False Negative (FN)	No security warning exists, and the function is vulnerable

Precision

Precision measures the proportion of correctly reported vulnerabilities among all vulnerabilities reported by a tool. A high precision score indicates that the tool generates few false alarms, making the results reliable and actionable. On the other hand, a low precision score shows that the tool often flags non-issues as vulnerabilities, reducing trust in its results. Precision does not account for how many of the possible vulnerabilities are marked.[27] Precision is calculated using the following equation:

$$Precision = \frac{TP}{TP + FP} \quad (2.1)$$

Recall

Recall, also known as sensitivity, evaluates the ability of the tool to detect vulnerabilities by comparing the number of found vulnerabilities to the total number of vulnerabilities. A high recall indicates that the tool is effective at minimizing missed vulnerabilities [27]. Recall is calculated as:

$$Recall = \frac{TP}{TP + FN} \quad (2.2)$$

¹¹<https://docs.Snyk.io/scan-with-Snyk/Snyk-code>

¹²<https://docs.Snyk.io/scan-with-Snyk/Snyk-code/Snyk-code-security-rules>

F1-score

F1-score measures the overall effectiveness of a SAT by combining precision and recall into a single metric using the harmonic mean. It is beneficial when FP and FN matter, as in real-world scenarios. A high F1 score means the tool detects many actual vulnerabilities (high recall) while keeping its reports accurate (high precision).[27]

$$F1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.3)$$

Pair-wise Prediction

Pair-wise prediction evaluates tools' accuracy based on their ability to correctly differentiate between vulnerable and benign versions of the same function. This approach is especially valuable for detecting whether tools are identifying the root cause of the vulnerability or simply reacting to surrounding patterns.

This evaluation method was initially developed for assessing Large Language Models (LLMs) using the PrimeVul dataset [8], but it is equally applicable to SATs. Table 2.2 shows a summary of pair-wise classification categories.

Table 2.2: Pair-wise classification categories for functions in the PrimeVul dataset.

Prediction	Explanation
Pair-wise, Correct Prediction	Correctly classifies the labels for both functions.
Pair-wise Vulnerable Prediction	Incorrectly classifies both functions as vulnerable.
Pair-wise Benign Prediction	Incorrectly classifies both functions as benign.
Pair-wise Reversed Prediction	Incorrect and inversely classify both functions.

Matthews Correlations Coefficient

The Matthews Correlation Coefficient (MCC) is a comprehensive metric considering all four fundamental metrics: TP, FP, TN, and FN. Unlike other metrics, MCC includes TN, making it particularly useful for evaluating classifiers on imbalanced datasets, such as Juliet, where benign samples vastly outnumber vulnerable ones.

The MCC ranges from -1 (complete disagreement) to +1 (perfect prediction), with zero indicating no better than random guessing. While commonly used in machine learning, MCC has also been applied to evaluate SATs [5, 15]. However, prior work has shown that MCC can produce saw-tooth effects when analyzing partial derivatives over TP and TN values, especially in uneven data distributions where the dataset contains very few vulnerabilities compared to benign functions [30]. Despite its imperfections, MCC offers valuable insight into tool performance when both positive and negative predictions matter.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (2.4)$$

2.1.4 Datasets

This study uses two complementary datasets, Juliet and PrimeVul. The datasets were selected during a literature review, and based on scale, earlier research and availability.

The study aimed to evaluate large amounts of real-world data, and, to no surprise, there are not many large collections of non-synthetic, real-world vulnerabilities openly available on the Internet. To reach the scale required for this thesis and to obtain fair, reliable results, the

dataset had to contain known, real vulnerabilities from earlier versions of well-known open-source projects. Such datasets are scarce, but a few have emerged recently with the rise of AI and LLMs, all created to train these models for vulnerability detection.

PrimeVul is one of these large, state-of-the-art collections: even companies such as Google use it to evaluate their AI. Other large vulnerability datasets, such as Big-Vul [11], use automatic vulnerability labeling which is highly inconsistent. Instead, the vulnerabilities in PrimeVul are selected and verified by a team of security analysts, leading to more accurate labeling compared to automatic alternatives. The PrimeVul dataset was released as recent as last year, with updates being made continuously. Its accompanying paper has been included in well-known conferences such as ICSE¹³, and it clearly explains the weaknesses of popular similar datasets [8].

Because no prior SAT research has evaluated PrimeVul, the study also needed a benchmark to confirm that the chosen techniques and methods remain consistent with earlier work. The Juliet Test Suite meets this requirement perfectly: it is large enough for the study, openly available, and expressly designed for static-analysis research. No other dataset offers the same solid structure, scale, and frequent appearance in prior studies.

Juliet

Juliet, a synthetic benchmark test suite released by the National Institute of Standards and Technology (NIST), contains thousands of short C/C++ functions in which the vulnerabilities are deliberately planted and fully labeled with their CWE identifiers (see Section 3.1.1)[2].

PrimeVul

To add real-world complexity, PrimeVul was used, which was developed for benchmarking machine learning models to find vulnerabilities in source code and based on open-source projects, where each vulnerable function is matched with the commit that fixes it, for a total of 4 659 “before-and-after” examples [8] (Section 3.1.2). The word historical data is sometimes used instead of realistic data to describe datasets based on past vulnerabilities. Together, the synthetic clarity of Juliet and the realism of PrimeVul provide both a clean baseline and a practical evaluation ground.

2.1.5 MITRE CWE Top 25

The MITRE CWE Top 25 is an annual ranking of the most critical software weaknesses based on real-world exploit data and community input¹⁴. This list is widely used in industry and research, highlighting the types of vulnerabilities most frequently found and exploited in software.

2.2 Related work

The field of static analysis for vulnerability detection has been extensively studied. This section reviews prior research on the effectiveness of SATs, their evaluation on both synthetic and real-world datasets, and the integration of machine learning in vulnerability detection. It also identifies key gaps in the current literature.

¹³<https://conf.researchr.org/home/icse-2025>

¹⁴<https://cwe.mitre.org/top25/>

2.2.1 SATs on Synthetic Projects

Previous research on SATs has primarily focused on synthetic datasets [10, 18]. These datasets consist of artificially constructed code samples that contain deliberately inserted vulnerabilities. For instance, the Juliet Test Suite is fabricated to illustrate specific vulnerability patterns with 64 000 C/C++ functions spanning 118 CWE classes [2]. Software like LAVA [9] and STONESOUP¹⁵ instead inject vulnerabilities into existing code, mixing synthetic vulnerable code into realistic codebase.

Gentsch [15] extensively evaluated open-source Static Application Security Testing (SAST) tools for detecting vulnerabilities in C code. The study used the Juliet Test Suite and benchmarked popular tools, including Clang-Tidy, Cppcheck, and Flawfinder, examining metrics such as precision, recall, and overall accuracy. The analysis found significant variability in tool performance based on different fault patterns.

Cohen et al. [6] describe the ideal test suite as one that satisfies three key criteria: it is similar to real-world production software, has complete knowledge of all vulnerabilities present, and includes diverse types of bugs in varied contexts. Juliet fulfills the latter two parts effectively. However, Juliet’s test cases are significantly smaller and simpler than production software. Therefore, another dataset is needed for an accurate representation of real-world complexity.

2.2.2 SATs on Real-World Projects

Although synthetic datasets enable systematic experimentation, they often lack the complexity and variability of real-world software. Several researchers have evaluated the performance of SATs in these environments. For instance, Lipp et al. [23] assessed six different tools on known and validated Common Vulnerabilities and Exposures (CVE) and concluded that they were relatively ineffective, the most accurate tool detecting only 53% of the vulnerabilities in their dataset, which is based on 192 vulnerabilities across 27 projects. Similarly, Choaroenwet et al. [3] found that individual tools correctly identified vulnerabilities in approximately 52% of vulnerability-containing commits.

Pereira and Vieira [7] analyzed the Mozilla open-source project where Cppcheck and Flawfinder were compared. The fundamental metrics were based on a file level, meaning if any vulnerability was found in the correct file, the whole file was counted as vulnerable. Additionally, the study reports the total number of alerts each tool generates across the project. The dataset used in this evaluation contains 2 441 known vulnerabilities, serving as the ground truth for measuring detection performance.

Massacci and Nguyen [24] analyzed previous work on the Mozilla project, focusing on how the choice of dataset and ground truth influences the outcome of a vulnerability detection study. Their findings show that different datasets and ground truth can yield varying results even when analyzing the same software project.

The study by Beller et al. [1] showed that 77% of open-source projects use some automatic SAT, and their manual analysis shows that 59% use automatic SAT. It gives some insight into the popularity of SATs and their popularity in open-source production pipelines.

2.2.3 Research gap

Existing evaluations of SATs fall short in two categories. First, dataset scope: Real-world studies usually analyze a single project or a handful of repositories, while synthetic-suite studies cover many samples but lack real-code diversity. Second, metric coverage: Many

¹⁵<https://www.iarpa.gov/research-programs/stonesoup>

papers use unbalanced datasets, which can skew the perceived effectiveness of the tool's performance.

To close this gap, our work benchmarks rule-based and machine learning-based SATs side-by-side on synthetic (Juliet) and large, labeled real-world vulnerabilities (PrimeVul) and reports a range of metrics, including MCC and pair-wise measures, to provide a balanced assessment of tool performance.



3 Method

This chapter introduces the method used to answer the research question, initially the two datasets: Juliet Test Suite and PrimeVul are introduced. Next, it presents the tools selected for analysis and a detailed workflow for collecting results. Then, it explains the evaluation metrics used to measure tool performance and concludes with a discussion of limitations and considerations.

3.1 Datasets

This section describes the two datasets used to evaluate and compare the SATs.

3.1.1 Juliet Test Suite

The Juliet Test Suite, developed by NIST, includes test files containing vulnerable and benign functions. To identify vulnerable code, the function name contains *bad* whilst benign functions contain *good*. The vulnerable functions closely resemble their benign counterparts. Table 3.1 summarizes the amount of CWEs, test cases, and files in Juliet [2].

Table 3.1: High-level statistics for the Juliet 1.3 C/C++ test suite.

Statistic	Amount
CWEs	118
Test Cases	64 099
Benign Functions	241 876
Vulnerable Functions	107 220
Files	105 416

Evaluating the tools on synthetic data provides an early view of their capabilities and hints at how the underlying technologies might scale to real-world code. Juliet has fully documented benign and vulnerable functions, illustrated in Figures 3.1 and 3.2, where the difference between the vulnerable and benign code is shown in bold for readability.

```

1 void CWE121_Stack-Based_Buffer_Overflow__char_type_overrun_memcpy_01_bad()
2 {
3     charVoid structCharVoid;
4     structCharVoid.voidSecond = (void *)SRC_STR;
5
6     /* Print the initial block pointed to by structCharVoid.voidSecond */
7     printLine((char *)structCharVoid.voidSecond);
8
9     /* FLAW: sizeof(structCharVoid) overwrites pointer voidSecond */
10    memcpy(structCharVoid.charFirst,
11           SRC_STR,
12           sizeof(structCharVoid));
13
14    structCharVoid.charFirst[sizeof(structCharVoid.charFirst) - 1] = '\0';
15    printLine((char *)structCharVoid.charFirst);
16    printLine((char *)structCharVoid.voidSecond);
17 }

```

Figure 3.1: Example of a vulnerable function from the Juliet Test Suite.

```

1 void good1() {
2     charVoid structCharVoid;
3     structCharVoid.voidSecond = (void *)SRC_STR;
4
5     /* Print initial block pointed by structCharVoid.voidSecond */
6     printLine((char *)structCharVoid.voidSecond);
7
8     /* FIX: Correct use of sizeof */
9     memcpy(structCharVoid.charFirst,
10           SRC_STR,
11           sizeof(structCharVoid.charFirst));
12
13    structCharVoid.charFirst[sizeof(structCharVoid.charFirst) - 1] = '\0';
14    printLine(structCharVoid.charFirst);
15    printLine((char *)structCharVoid.voidSecond);
16 }

```

Figure 3.2: Example of a benign function from the Juliet Test Suite.

3.1.2 PrimeVul

The paired structure of PrimeVul enables a direct comparison between pre- and post-fix code. The vulnerable and benign functions are closely related, differing only by the vulnerability fix. In Figure 3.3 and Figure 3.4, examples of such function pairs are presented where the vulnerability is found in line 4 and bolded in the figures.

```

1 u32 GetHintFormat (GF_TrackBox *trak)
2 {
3     GF_HintMediaHeaderBox *hmhd = (GF_HintMediaHeaderBox
4         *)trak->Media->information->InfoHeader;
5     if (hmhd->type != GF_ISOM_BOX_TYPE_HMHD)
6         return 0;
7
8     if (!hmhd || !hmhd->subType) {
9         GF_Box *a = (GF_Box*)gf_list_get(trak->Media->information->
10             sampleTable->SampleDescription->child_boxes, 0);
11         if (!hmhd) return a ? a->type : 0;
12         if (a) hmhd->subType = a->type;
13         return hmhd->subType;
14     }
15     return hmhd->subType;
16 }

```

Figure 3.3: Example of a vulnerable function from PrimeVul.

```

1 u32 GetHintFormat (GF_TrackBox *trak)
2 {
3     GF_HintMediaHeaderBox *hmhd = (GF_HintMediaHeaderBox
4         *)trak->Media->information->InfoHeader;
5     if (!hmhd || (hmhd->type != GF_ISOM_BOX_TYPE_HMHD))
6         return 0;
7
8     if (!hmhd || !hmhd->subType) {
9         GF_Box *a = (GF_Box *)gf_list_get(trak->Media->information->
10             sampleTable->SampleDescription->child_boxes, 0);
11         if (!hmhd) return a ? a->type : 0;
12         if (a) hmhd->subType = a->type;
13         return hmhd->subType;
14     }
15     return hmhd->subType;
16 }

```

Figure 3.4: Example of a benign function from PrimeVul.

3.2 Workflow

This section outlines the full workflow for evaluating each SAT on both the Juliet Test Suite and the PrimeVul dataset. The process begins with dataset preparation, followed by running the tools on the codebases. Next, the raw outputs are filtered and parsed to a standardized format. The processed data is then used for classification, where findings are labeled based on their correctness, and finally, the results are evaluated using established metrics. Figure 3.5 illustrates this step-by-step pipeline.

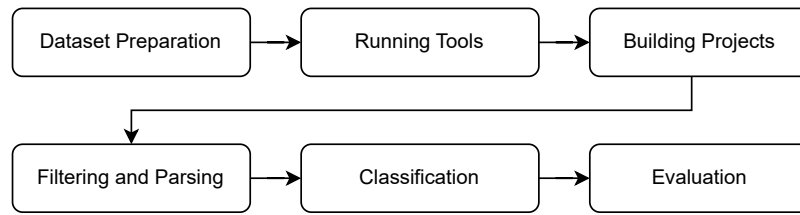


Figure 3.5: Workflow diagram for evaluating SATs.

3.2.1 Dataset Preparation

The only preparation needed for the Juliet Test Suite was running the provided script for building the project. Since PrimeVul was created for machine learning, the data is already split into test, training, and validation data containing 4 704 vulnerable-benign pairs. A script was initially run to combine the subsets into a standard, larger dataset. The script also gathers the commit ID for the vulnerable commit, the size of the repository, and the full file path to the vulnerable file. If any of these could not be gathered, the pair was no longer deemed relevant since a comparison could not be run and was thus removed from the dataset. After the initial filtering, 6 640 data points remained, resulting in 3 320 data pairs. A second filtering step was applied to eliminate duplicates, verify matching file names and paths, and extract the start and end lines for each function. This process resulted in 2 985 function pairs across 468 distinct projects.

All 1 206 datapoints based on the Linux repository were removed because the project was too big for the tools to analyze within a reasonable time.

3.2.2 Running Tools

The first step was running the SATs on Juliet Test Suite and the realistic data and capturing their raw outputs. Automation scripts were developed for each tool to simplify this process. Some tools, such as Clang-Tidy, require the project to be built. This is a bigger process for the realistic data that could be read more about in Section 3.2.3.

The scripts for Juliet were iterated through each CWE-specific directory, and for PrimeVul, they were iterated through each project. Flags and arguments were configured to ensure fairness and comparability among the tools, providing severity levels and generating machine-readable output formats. Which flags were used and arguments for each tool are presented in Table 3.2. Output-related flags have been excluded from this table to maintain focus on the configuration parameters and increase the table's readability.

Due to the limited amount of runs for Snyk, datapoints that were analyzed by snyk were restricted to combinations where all other tools included in the comparison successfully executed on the corresponding data points. This ensures consistency in the evaluation and avoids skewing the results due to missing tool outputs.

Table 3.2: Tools and their corresponding flags used in the analysis.

Tool	Flags	Docs
Cppcheck	<code>-template="file,line,severity,message"</code>	1
Flawfinder	<code>-error-level=3 -sarif</code>	2
Clang-Tidy	<code>-checks=*,clang-analyzer-security*</code>	3
Semgrep	<code>scan -config p/c -pro -json</code>	4
Snyk	<code>code test</code>	5

3.2.3 Building Projects

Running Clang-Tidy on a project requires a compilation database. In this case, a `compile_commands.json` file was used⁶. The tool Bear generated compilation databases automatically during the project build process⁷. This provided a uniform method for generating compilation databases for projects built using Automake, CMake, or standard Makefiles.

Clang-Tidy uses the compilation database to analyze the C++ AST of each project. To address the challenge of building 468 projects, a script was developed to automate the build process using the standard build systems Make, CMake, and Autotools. The process for each data point follows the steps as seen in Figure 3.6.

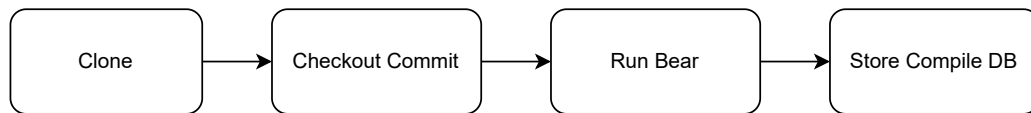


Figure 3.6: Workflow diagram for building compile database.

If the project were built or could not reasonably be built, the data point would be considered done, and the process for that project would be finished. This would result either in a compilation database file or the data point being filtered away.

There were several reasons why some projects could not be built:

- The project depended on outdated libraries or submodules that no longer exist.
- The project required older versions of dependencies that are not backward-compatible with current versions.
- The source code contained constructs no longer accepted by modern compilers, such as deprecated hashing algorithms.

¹<https://Cppcheck.sourceforge.io/manual.pdf>

²<https://github.com/david-a-wheeler/Flawfinder/blob/master/README.md>

³<https://clang.llvm.org/extra/clang-tidy/>

⁴<https://Semgrep.dev/docs/cli-reference>

⁵<https://docs.Snyk.io/Snyk-cli/commands/code-test>

⁶<https://clang.llvm.org/docs/JSONCompilationDatabase.html>

⁷<https://github.com/rizotto/Bear>

- The project could not be built on a standard Linux Ubuntu system (e.g., projects depending on Windows-specific components like Mintty).
- Git submodules linked to the project no longer existed (e.g., the Atheme project).

Projects using build systems such as Python-based scripts, Meson, or Bazel were not included in these systems and served as the primary build entry point. These build systems were deemed unsuitable for automation, as there is no standardized or uniform method for consistently building projects across them. However, if such projects were integrated into a larger build system that provided an entry point through Make, CMake, or Autotools, they were included in the dataset.

3.2.4 Filtering and Parsing

Once all raw data was collected, it was transformed into a standardized format to enable consistent classification. A unified CSV schema was created, containing core fields common to all tools and necessary for gathering the metrics: file name, line number, and error message. While the analyzers often offered additional metrics, none were essential for classification and available across all tools, so they were excluded from this standardized format. Because each tool has different output structures and naming conventions, individual reformatting scripts were developed to parse and map the data into the following CSV format:

⟨File Location, Line, Error Message⟩

Additionally, Clang-Tidy was filtered to display only security-related alerts since we are not interested in its linting capabilities or syntax-based remarks. Similar filters were applied to the Flawfinder output, only including entries of "warning" severity or higher. The outcome from this section was one CSV file per tool containing the filtered and formatted results from the full C++ test suite.

3.2.5 Classification

The next step was mapping reported warnings back to Juliet's labeled functions. The output generated by the SATs does not generally reference function names directly. Instead, warnings are associated with line numbers. Therefore, line-to-function mapping was performed to determine which function each reported warning corresponded to. The evaluation was done by iterating over the CWE files and checking, on a function-by-function basis, whether a matching warning existed in the CSV file from each tool.

For PrimeVul, if a function is vulnerable, it is explicitly labeled within the data. Function-to-line mapping was run when setting up the data. The automation then checks for each data point to see if a result containing the vulnerable function exists. Each Juliet and PrimeVul data function was then classified according to Table 2.1.

Some limitations were implemented to avoid modifying the results by over-counting multiple alerts within the same function. If a single function was flagged multiple times by a tool at different line numbers, it was still counted only once as a positive for that function. Thus, if a vulnerable function contained multiple warnings, it contributed a single TP, while a benign function incorrectly flagged at multiple lines counted as one FP.

3.2.6 Evaluation

Automated scripts were developed to compute the evaluation metrics described in Section 2.1.3, based on the previously performed classification and filtering. Graphs and pie charts were created based on the scripts' results to illustrate the overall performance of the tools.

For the pair-wise score if one run out of the pair DNF, the entire pair is removed from the statistics since both vulnerable and benign version is needed to calculate pair-wise metrics.

3.3 Extended Data Processing

In addition to calculating the evaluation metrics above, additional processing was performed in order to see how tools perform on different types of CWES, and how tools can be combined for better results.

3.3.1 MITRE CWE Top 25

The Juliet Test Suite was filtered to evaluate tool performance on the most common vulnerabilities. Table 3.4 shows the top CWE categories used in the study. Some CWEs are missing because a few top vulnerabilities are related to databases, like SQL injection, or web, like Cross-Site Scripting. Our dataset contains neither of these vulnerability types; thus, they were excluded.

Table 3.4: CWE categories for "Top 25" analysis on Juliet.

CWE ID	Description
CWE78	OS Command Injection
CWE121	Stack-based Buffer Overflow
CWE122	Heap-based Buffer Overflow
CWE124	Buffer Underwrite
CWE126	Buffer Over-read
CWE127	Buffer Under-read
CWE190	Integer Overflow
CWE400	Uncontrolled Resource Consumption
CWE416	Use After Free
CWE426	Untrusted Search Path
CWE476	NULL Pointer Dereference

3.3.2 Cross-Tool Combination Strategies

To evaluate how different tools combinations can be used for better performance, two strategies were considered: intersections, which highlight consensus between tools, and unions, which aggregate all unique findings. By calculating new values for the advanced metrics such as precision, recall, F1-score and MCC, these strategies offer insight into how multiple tools can be leveraged to support more effective secure development workflows.

Intersections

In addition to traditional evaluation metrics, the classification intersections for various tool combinations were computed. This approach explores whether combining multiple tools can improve overall accuracy by "filtering out" incorrect classifications from individual tools. The intersection eliminates classifications only found by singular tools while accepting more commonly identified vulnerabilities detected by multiple tools. This is the same as doing a logical AND with the output results.

Intersections between tool results were calculated by iterating over the entire test suite, covering all CWEs and functions, and isolated entries where multiple tools reported the same file, classification, and function. For PrimeVul, intersection data was computed showing how many datapoints the different tools were successfully run on, giving some insight into how easily the tools run on any given project.

Unions

Similarly, classification unions were computed to evaluate whether combining tool outputs could increase overall detection coverage. In contrast to intersections, unions consider any vulnerability reported by at least one tool, thereby accepting broader warnings. This strategy aims to maximize coverage, even at the potential cost of introducing more FPs. This is the same as applying a logical-OR across tool outputs.

Union analysis followed the same methodology as the intersection analysis but included all unique findings from the participating tools rather than only shared ones. This comparison helps to understand how tool combinations can complement one another to provide a more comprehensive security assessment.

4 Results

This section presents the results of evaluating the SATs. The evaluation is divided into two major parts: the first part focuses on the Juliet Test Suite, and the second part focuses on PrimeVul.

4.1 Performance on Juliet

This section presents the results obtained from evaluating SATs on the Juliet Test Suite, getting data for the first research question in Section 1.3. The tools chosen were Cppcheck, Flawfinder, Clang-Tidy, Semgrep, and Snyk.

4.1.1 Classification Results

Table 4.1 presents the classification performance of all tools on the full Juliet dataset using the method described in Section 3.2.5.

Table 4.1: Classification results for SATs on the Juliet Test Suite.

Tool	TP (%)	FP (%)	TN (%)	FN (%)
Clang-Tidy	4.6	9.3	60.0	26.1
Flawfinder	2.0	3.2	66.1	28.7
Cppcheck	1.9	0.9	68.4	28.8
Snyk	1.7	1.0	68.2	29.0
Semgrep	0.5	0.2	69.0	30.2

The results show notable variation between tools. Clang-Tidy achieved the highest TP rate (4.6%) but also the highest FP rate (9.3%). Semgrep had the lowest TP rate (0.5%) and FP rate (0.2%). TN and FN values were consistently high across all tools.

MITRE Top 25

The results of the tools' performance on the 25 most common CWEs, as described in Section 3.3.1 can be found in Table 4.2.

Table 4.2: Classification results for SATs MITRE top 25 CWEs.

Tool	TP (%)	FP (%)	TN (%)	FN (%)
Clang-Tidy	11.9	11.9	19.6	56.5
Flawfinder	2.7	4.5	63.6	29.2
Cppcheck	2.1	1.3	66.8	29.9
Snyk	1.5	0.9	67.1	30.5
Semgrep	0.3	0.1	68.0	31.6

Figure 4.1 shows the distribution of TP, TN, FP and FN classifications reported across all tools for the full Juliet Test Suite, percentages total 99.9% due to rounding errors. Figure 4.2 shows the distribution for the MITRE top 25 subset.

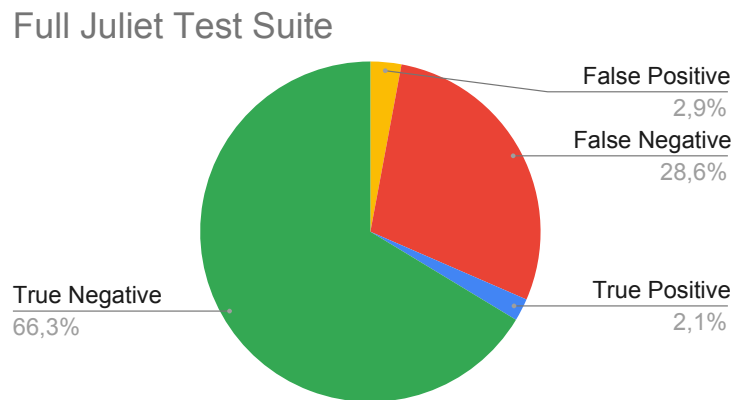


Figure 4.1: Classification distribution on Juliet Test Suite.

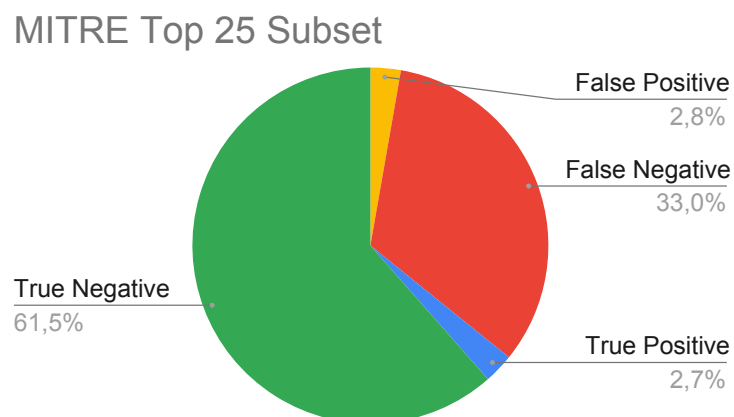


Figure 4.2: Classification distribution on MITRE top 25 subset.

4.1.2 Advanced Metrics

Based on the classification results, additional performance metrics were calculated for each tool based on the Equations 2.1, 2.2 and 2.3. The respective values for each tool can be found in Table 4.3.

Table 4.3: Precision, Recall, F1-score and MCC on the Juliet Test Suite.

SAT	Precision (%)	Recall (%)	F1-score (%)	MCC (-1 – +1)
Clang-Tidy	32.9	14.9	20.5	0.02
Flawfinder	38.3	6.4	11.0	0.04
Cppcheck	68.5	6.2	11.4	0.14
Snyk	62.6	5.7	10.4	0.12
Semgrep	67.1	1.6	3.1	0.07

Intersection Data

The results when analyzing the overlap between tools as described in Section 3.3.2 are found in Table 4.4.

Table 4.4: Intersection counts for TP and FP on the Juliet dataset.

Combination	TP	FP
Pairs		
Snyk, Semgrep	967	557
Snyk, Cppcheck	440	147
Snyk, Clang	1705	1927
Snyk, Flawfinder	675	886
Semgrep, Cppcheck	365	40
Semgrep, Clang	154	138
Semgrep, Flawfinder	132	159
Cppcheck, Clang	1376	1102
Cppcheck, Flawfinder	406	318
Clang, Flawfinder	2595	4962
Triplets		
Snyk, Semgrep, Cppcheck	281	0
Snyk, Semgrep, clang	44	45
Snyk, Semgrep, Flawfinder	58	66
Snyk, Cppcheck, Clang	36	49
Snyk, Cppcheck, Flawfinder	36	49
Snyk, Clang, Flawfinder	595	834
Semgrep, Cppcheck, Clang	18	0
Semgrep, Cppcheck, Flawfinder	18	0
Semgrep, Clang, Flawfinder	56	64
Cppcheck, Clang, Flawfinder	222	162
Quadruples		
Snyk, Semgrep, Cppcheck, Clang	0	0
Snyk, Semgrep, Cppcheck, Flawfinder	0	0
Snyk, Semgrep, Clang, Flawfinder	20	33
Snyk, Cppcheck, Clang, Flawfinder	36	49
Semgrep, Cppcheck, Clang, Flawfinder	18	0
All Five		
Snyk, Semgrep, Cppcheck, Clang, Flawfinder	0	0

Union data

The results of union data can be found in Appendix B.

4.2 Performance on PrimeVul

This section presents the results obtained from evaluating SATs on the realistic data, answering the second research question presented in Section 1.3.

4.2.1 Classification Results

A total of 4 764 datapoints were used for the data gathering after the method described in Section 3.2.1 were completed. For some of the tools the data gathering Did Not Finish (DNF), the ratio between DNFs and successful runs each tool have can be seen in Table 4.5.

Table 4.5: Successful run and DNF rates for SATs on the PrimeVul dataset.

Tool	Successful Runs (%)	DNF (%)
Clang-Tidy	47.42	52.58
Flawfinder	83.47	16.53
Cppcheck	85.90	14.10
Snyk	99,04	0.96
Semgrep	88.33	11.67

Table 4.6 summarizes the classification performance of all tools on the realistic dataset using the previously described method in Section 3.2.5. These results are based after excluding the DNF.

Table 4.6: Percentage distribution of classification results for SATs and LLMs on the PrimeVul dataset.

Tool	TP (%)	FP (%)	TN (%)	FN (%)
Clang-Tidy	2.30	2.21	47.70	47.79
Flawfinder	1.31	1.28	48.69	48.72
Cppcheck	0.32	0.34	49.63	49.71
Snyk	1.08	1.15	49.01	48.76
Semgrep	0.29	0.24	49.76	49.71

As seen in Table 4.6 a clear majority of datapoints are TN and FN meaning the tools didn't produce a finding within the function the datapoint is representing. The total distribution of TP, TN, FP and FN classifications reported across all tools can be seen in Figure 4.3. Figure 4.3 percentages add up to 100.1% due to rounding error.

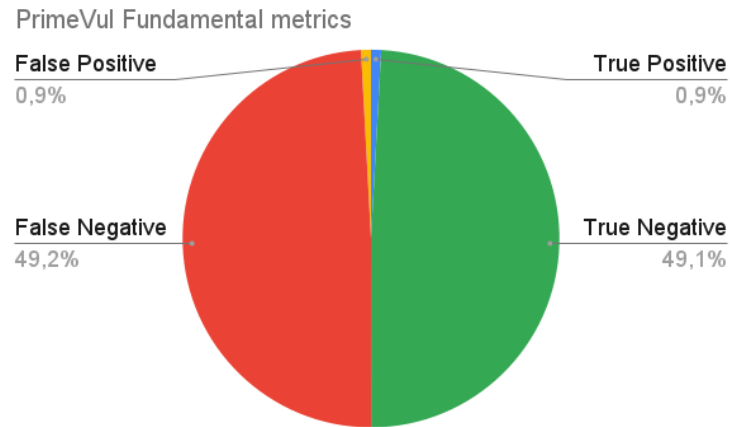


Figure 4.3: SAT Classification distribution on PrimeVul.

4.2.2 Advanced Metrics

From the classification results, additional performance metrics were calculated based on the Equations 2.1, 2.2, 2.3 and 2.4. The results can be seen in Table 4.7

Table 4.7: Precision, Recall, F1-score, and MCC on the realistic dataset.

Tool	Precision (%)	Recall (%)	F1-score (%)	MCC (-1 – +1)
Clang-Tidy	51.0	4.6	8.4	0.004
Flawfinder	50.5	2.6	5.0	0.002
Cppcheck	48.1	0.6	1.3	-0.003
Snyk	48.6	2.2	4.2	-0.004
Semgrep	54.5	0.6	1.1	0.007

The pair-wise score is calculated according to Section 2.1.3. Sometimes one run out of the pair DNF, in this case that entire pair is removed from the statistics according to Table 4.8.

Table 4.8: Pairwise successful run and Did Not Finish (DNF) rates for SATs on the PrimeVul dataset.

Tool	Successful Runs (%)	DNF (%)
Clang-Tidy	96.7	3.3
Flawfinder	97.7	2.3
Cppcheck	97.8	2.2
Snyk	99.8	0.2
Semgrep	98.5	1.5

The values for the Pair-Wise (P-W) metrics in percentages *Correct Prediction (CP)*, *Vulnerable Prediction (VP)*, *Benign Prediction (BP)* and *Reversed Prediction (RP)* after all pairs that DNF can be found in Table 4.9

Table 4.9: Pair-wise Correct Prediction, Vulnerable Prediction, Benign Prediction and Reversed Prediction on the realistic data.

Tool	P-W CP (%)	P-W VP (%)	P-W BP (%)	P-W RP (%)
Clang-Tidy	0.3	4.2	95.4	0.1
Flawfinder	0.2	2.4	97.3	0.1
Cppcheck	0.0	0.6	99.3	0.1
Snyk	0.0	2.1	97.7	0.1
Semgrep	0.2	0.4	99.3	0.1

Intersection Data

For the PrimeVul dataset, the overlap in tool reports were investigated to analyze how frequently different tools identified the same vulnerable or benign functions. This analysis was conducted across all tool combinations to assess agreement between tools.

Table 4.10 presents the total number of intersections for each tool combination, representing cases where multiple tools flagged the same function as vulnerable. Intersections were counted regardless of whether the detection was correct or not.

Table 4.10: Intersection counts for TP and FP on PrimeVul dataset.

Combination	TP	FP
Single Tools		
Snyk	17	18
Clang-Tidy	52	50
Cppcheck	13	14
Flawfinder	52	51
Semgrep	12	10
Pairs		
Snyk, Clang-Tidy	13	14
Snyk, Cppcheck	3	3
Snyk, Flawfinder	11	11
Snyk, Semgrep	6	6
Clang-Tidy, Cppcheck	5	6
Clang-Tidy, Flawfinder	25	25
Clang-Tidy, Semgrep	7	7
Cppcheck, Flawfinder	7	8
Cppcheck, Semgrep	3	2
Flawfinder, Semgrep	9	8
Triplets		
Snyk, Clang-Tidy, Cppcheck	3	3
Snyk, Clang-Tidy, Flawfinder	10	11
Snyk, Clang-Tidy, Semgrep	5	5
Snyk, Cppcheck, Flawfinder	3	3
Snyk, Cppcheck, Semgrep	1	0
Snyk, Flawfinder, Semgrep	5	5
Clang-Tidy, Cppcheck, Flawfinder	3	4
Clang-Tidy, Cppcheck, Semgrep	1	0
Clang-Tidy, Flawfinder, Semgrep	7	7
Cppcheck, Flawfinder, Semgrep	2	1
Quadruplets		
Snyk, Clang-Tidy, Cppcheck, Flawfinder	3	3
Snyk, Clang-Tidy, Cppcheck, Semgrep	1	0
Snyk, Clang-Tidy, Flawfinder, Semgrep	5	5
Snyk, Cppcheck, Flawfinder, Semgrep	1	0
Clang-Tidy, Cppcheck, Flawfinder, Semgrep	1	0
All Five		
Snyk, Clang-Tidy, Cppcheck, Flawfinder, Semgrep	1	0

Union data

The results of Union data can be found in Appendix B.



5 Discussion

This section will discuss the results presented in Section 4 and the method used to gather those results described in Section 3. Finally, the findings are considered in a broader context.

5.1 Results

This chapter discusses the findings from Chapter 4. This discussion is structured around the research questions, first focusing on the Juliet Test Suite and then examining the PrimeVul dataset results. Subsequently, tool-specific observations will be addressed, integrating the results from both datasets. Finally, the method used to obtain the results will be reviewed, focusing on the challenges encountered and the measures taken to mitigate their impact on the findings.

5.1.1 Metrics

Many different metrics are used to analyze the data; it is important to know what each metric does and how it applies in the security context.

Fundamental metrics

The fundamental metrics of TP, FP, TN, and FN, as described in Chapter 2.1.3, are mainly used to calculate the other metrics that give insight into the tool's performance.

Precision

This metric gives us value in security context where FPs can lead to alert fatigue. Tools that only raise warnings when they are very confident may achieve high precision while still missing many probable vulnerabilities, resulting in less secure code. Precision can give a skewed impression of tool performance if considered in isolation, especially on imbalanced datasets where FNs are not penalized.

Recall

Precision is a metric useful in security evaluations where missing vulnerabilities could have severe consequences. However, precision considered in isolation can also give skewed performance since it does not penalize FPs, which could result in alert fatigue.

F1-score

Since F1-score combines precision and recall, it gives a more balanced view of tool performance when both missing vulnerabilities and generating too many false alarms are important. However, because F1-score ignores TNs entirely, it can skew results on imbalanced datasets where TNs and FPs are few.

Pair-wise metrics

Pair-wise metrics are valuable when identifying data points that could benefit from deeper manual analysis. For instance, pair-wise reversed predictions, where the tool flags the patched function but misses the vulnerable one, can point to major issues in tool logic or rule coverage.

Matthews Correlation Coefficient

MCC is a metric that is useful for both balanced and unbalanced datasets; the main advantage over other advanced metrics is that MCC takes TN into account, which no other tool directly does; if the dataset is balanced, precision and F1-score would indirectly take it into account since it takes FPs into account which is the inverse [5, 15]. MCC has flaws; a saw-tooth pattern will appear when analyzing the first-order partial derivate concerning TP for MCC on balanced and imbalanced datasets. The shape of the saw-tooth changes based on the value of TN. The same saw-tooth effect is seen when deriving for TN and changing the value of TP [30]. This concludes that MCC is not a perfect metric to evaluate any dataset, but for this study, since FN is evaluated. We have one balanced dataset (1:1) with PrimeVul pairs and one imbalanced (1:2) with Juliet. The saw-tooth patterns are very similar, and there is still much value in looking at the MCC.

5.1.2 Juliet Test Suite Results

The Juliet Test Suite serves as a reliable baseline for validating our methodology. Due to its clearly labeled functions enable verification that each SAT is properly configured and that our post-processing scripts correctly capture, filter, and classify findings as intended[2].

Additionally, using a well-known and researched test suite allows us to cross-reference our results with earlier Juliet studies, providing a reliability check for the rest of the evaluation before using similar methods on the more challenging PrimeVul dataset.

The classification Table 4.1 shows a large amount of TN overall. This is expected for multiple reasons; firstly, the classifications were made per function as described in Section 3.2.5, and most functions in the test suite are benign. Furthermore, Figure 4.1 shows that only about 5% of functions were flagged. Given that few flagged functions are relative to a largely benign dataset, a high number of TN is expected.

Cppcheck, Snyk, and Semgrep follow a pattern of about twice the number of TPs compared to FPs; this pattern is then reversed for Clang-Tidy and Flawfinder, which follow the opposite pattern of nearly double the FPs compared to their TPs, along with a higher count of flagged functions. This indicates that they are more aggressive when detecting potential vulnerabil-

ities. In tradeoff for the worse precision, a higher recall score is found for Clang-Tidy and Flawfinder.

Overall, the tools were largely inefficient at finding vulnerabilities in the synthetic dataset, identifying only about 7% of the intentionally vulnerable functions.

MITRE Top 25

As shown in Figure 4.1, the tools had a comparable performance on the most common CWEs as they did across the entire dataset. This observation is interesting, as one might expect enhanced performance on more widely recognized vulnerabilities. However, the TP rate saw only a slight increase of 0.5 percentage points, while the FN rate increased by 4.6 percentage points, suggesting that the tools performed slightly worse on the vulnerabilities in MITRE Top 25.

Advanced Metrics

Table 4.3 presents the precision, recall, F1-scores, and MCC for each tool. These metrics provide a more balanced view of performance than raw classification counts, as they highlight the trade-offs between detecting genuine vulnerabilities (recall) and correctly identifying them (precision) [27].

Most tools report moderate to high precision, ranging from about 33% to 68%, but with low recall between 1.5% and 14.9%. This suggests that, while many of the flagged issues are correct, a large portion of actual vulnerabilities are still undetected. Practically, this means a high risk of potential security issues going unnoticed.

The F1-score is below 21% for all tools, indicating a low general performance for vulnerability detection on the Juliet Test Suite. The MCC gives us some insight by considering TPs, FPs, FNs, and, unique to this metric, TNs. Having a metric that takes all four of the basic metrics into account is extra important when analyzing an imbalanced dataset such as the Juliet Test Suite. The tools have a spread from 0.14 down to 0.02, giving quite a high variance in values; however, they are far away from a perfect prediction, which would be a value of 1. However, since all tools are above 0, they all have predictive capability. Due to the low numbers, the predictive capability is very close to random guessing.

Classification Intersections

Looking at Table 4.4, we see that although single-tool usage detects many vulnerabilities, it also produces a large number of FPs. The more tools producing the same classification should increase the likelihood of that classification being correct. Interestingly, this is only true for some cases. For example, the pair *Semgrep* and *Cppcheck* yields 365 TP but only 40 FPs, indicating high confidence when both tools agree. Adding *Snyk* to this pair results in 281 TPs and *no* FP. In other words, every vulnerability collectively flagged by Snyk, Semgrep, and Cppcheck was genuinely a genuine security flaw. On the other hand, Clang and Flawfinder had intersections with about twice as many FPs compared to TPs. Intersections of four or more tools are, as expected, quite rare; see Table 5.1.

Table 5.1: Summation of TP and FP counts across singles, pairs, and triplets.

Combination Type	Total TP	Total FP
Singles	37,067	50,803
Pairs	8,815	10,236
Triplets	1,364	1,269
Quadruples	74	82
All Five	0	0

Single tools report the most significant volume of detections but also the highest number of FPs. Moving to pairs and then triplets, the overall volume of detections declines rapidly as all vulnerabilities that are marked by only a single tool are discarded. This leads to the ratio of TPs to FPs improving somewhat. The best ratios are achieved when hand-picking a combination of tools from Table 4.4, which produces many TPs but filters out the FPs.

5.1.3 PrimeVul Dataset

In comparison to the Juliet Test Suite, we only used the pair-wise dataset from PrimeVul, leading us to have 50% benign and 50% vulnerable functions, with each function being before and after the patch correcting the vulnerability. The functions are based on realistic historical data, meaning the complexity of each function is similar to what would be expected in a real-world project. This is reflected in only 1.6% of functions being flagged. Due to the low amount of flagged functions, we deemed that further quantitative analysis of the results would not be meaningful. This means the MITRE Top 25 analysis was not performed, and no analysis based on the intersection counts results was performed.

Historic data

One reason for the poor results on PrimeVul could be due to a lot of open-source projects running some SAT before the change is accepted into the central repository, as described in Section 2.2.2. This could mean that the easier vulnerabilities are found and fixed before they are merged, resulting in only complex vulnerabilities making it past the automated tools, thus making the dataset only contain complex vulnerabilities. However, to enable a meaningful comparison and gain insight into real-world performance, this trade-off was accepted. Moreover, openly available, untested real-world code is scarce, largely because of understandable security concerns.

5.1.4 Tool-Specific Observations

This section discusses the specific characteristics and behaviors of each tool, aiming to connect their underlying techniques with the observed results. By analyzing their internal approaches and reported metrics, we can better understand the potential reasons behind their respective performances.

Clang-Tidy

Clang-Tidy’s AST-based approach enables it to flag a larger range of potentially vulnerable patterns, contributing to its highest number of TPs (15 996) among the tested tools. Using many checks results in broader coverage but introduces a large number of FPs (32 506), resulting in a relatively low precision (32.9%) and high recall (14.9%). Issues and questions related to FPs from Clang-Tidy are common, and they have a webpage providing fixes for

the most common ones¹. However, these fixes are mainly on a per-case basis and thus were not included in the study due to the large-scale nature of the dataset.

Clang-Tidy is especially interesting since it has been outperforming the other tools in precision, recall, and F1-score but is dead last in MCC with only 0.02. Since MCC is the only value accounting for a higher FP rate, this statistic is important in an unbalanced dataset.

Cppcheck

Cppcheck is focused on finding undefined behavior and potential security vulnerabilities in the code through a comprehensive list of checks². Cppcheck uses checks and unsound flow-sensitive analysis as described in Section 2.1.2, which resulted in 6 687 TPs.

Cppcheck does not perform exhaustive exploration, potentially causing missed vulnerabilities, but helps maintain a lower FP count of 3 073. Overall, this approach explains why Cppcheck flags fewer overall vulnerabilities compared to more aggressive or simpler pattern-matching tools, which trade detection for fewer false alarms. Cppcheck was run once without a compiled database and once with each data point due to the similar result when running the tool with a compiled database, as without it, it was excluded from the results section.

Flawfinder

Flawfinder uses a straightforward pattern-matching approach that allows rapid analysis but significantly contributes to its high FP count (11 104). The benign code is incorrectly flagged due to the lack of contextual understanding. Despite this limitation, Flawfinders simplicity means it requires minimal setup and still runs efficiently with little configuration which could make it useful in situations where there is little time to set-up a more complex SAT.

Semgrep

Semgrep marked a relatively small amount of functions (2 547) compared to other tested tools but mainly had correct marks as identified by its precision score of 67.1%. One potential reason that Semgrep flagged so few could be that the default rule set for C / C++ that Semgrep uses only has 53 rules³. The Semgrep team curates these rules and is quite targeted, leading to a better precision than most other tools with larger rule sets. Downloading and adding more rules could lead to more vulnerabilities being found while also increasing the amount of FPs.

Snyk

The combination of data-flow and control-flow analysis with predefined security rules could lead to the accurate detection of vulnerabilities, significantly reducing FPs, totaling 3 631. Relying on learned patterns and rule definitions also leads to missed vulnerabilities, with a TP count of 6 074. The Snyk analysis need to be analyzed especially close due to the limited amount of runs that were had meaning the entire dataset could not be analyzed.

5.1.5 Cross-tool combination strategies

In addition to the advanced metrics, the strategy of combining different tools with the set-based logical operations of intersections and unions.

¹<https://clang.llvm.org/docs/analyzer/user-docs/FAQ.html>

²<https://sourceforge.net/p/cppcheck/wiki/ListOfChecks/>

³<https://Semgrep.dev/p/c>

Intersections

To understand where tools agree or disagree, we looked at overlap in the tool results. Intersections based on TPs and FPs were calculated; these are found in Table 4.4.

The most interesting overlap occurs on the Juliet Test Suite when *Snyk*, *Semgrep*, and *Cppcheck* all agree: they jointly flag 281 functions, and every one of those reports is a TP. This suggests that, despite their different detection strategies, the three tools complement each other well. By contrast, other tool pairs show much smaller overlaps and often mix true and FPs. Generally, requiring multiple tools to flag the same functions quickly reduces the number of hits and rarely yields significant improvements in detection rates. Even the seemingly impressive intersection of 281 TPs is bad given the 64 000+ vulnerable functions in the Juliet 1.3 C++ test suite (Table 3.1). As mentioned before, due to the low amount of flagged functions for the PrimeVul data, no analysis can be done on the intersection data.

Unions

Due to the volatile results, a valuable discussion and conclusion could not be drawn based on the union data.

5.1.6 Comparison with earlier work

Looking at the overall distribution in Figure 4.3, there is a relatively large number of FN (28,6%), indicating a large number of vulnerabilities were not detected by the SATs. Comparing our results with earlier work by Gentsch [15], we observe some notable differences in tool performance on the Juliet dataset, as seen in Table 5.2.

Table 5.2: Precision and recall on the Juliet dataset compared with values reported by Gentsch.

Tool	Prec. (% , G20)	Recall (% , G20)	Prec. (% , Ours)	Recall (% , Ours)
Clang-Tidy	9	17	32.9	14.9
Cppcheck	28	5	68.5	6.2
Flawfinder	20	9	38.3	1.6

When compared to earlier results, our precision results are significantly higher. This could be due to methodological differences. Gentsch only counted TP when a tool identified the exact line number of the vulnerability, whereas our classification was done on a per-function basis, as described in Section 3.2.5. Per-line evaluation, used only for the Juliet Test Suite, offers higher precision and is ideal when exact ground truth is available. However, it may penalize tools that correctly detect the vulnerable function without matching the exact line. Per-function classification is more forgiving and better reflects practical usage, especially in larger codebases where vulnerabilities span multiple lines or precise labels are unavailable.

5.1.7 Large Language Models & AI

In addition to SATs, this study also evaluated the performance of two prominent LLMs on vulnerability detection. The methodology and results are described in more detail in Appendix A. However, these findings were excluded from the central part of the report.

We chose not to directly compare the language models with the SATs, as their approaches differ significantly. For instance, SATs are given full context, analyzing entire projects or files where vulnerabilities occur. They also output information in addition to classifying vulnerabilities, such as type, affected lines, and severity. In contrast, the language models were given only isolated function strings to classify as vulnerable or not. This limited input reflects the

structure of the PrimeVul dataset, which is designed for machine learning. Furthermore, giving LLMs a broader context quickly leads to practical constraints related to model size and computational resources.

Additionally, the PrimeVul dataset is split into training, validation, and test sets, and these splits are used to fine-tune the models. As a result, less data is available for final evaluation compared to the static tools, which can be tested on the entire set of benign and vulnerable function pairs.

These substantial differences in context, data availability, evaluation scope, and methodology make a direct comparison between LLMs and SATs unfair.

5.2 Method

5.2.1 Replicability

Ensuring reproducibility was a central focus of this study, leading to the development of numerous scripts. These scripts range from large-scale automation tools for data collection to smaller scripts that handle tasks that could be done manually, such as merging training, test, and validation data into a single file. Since our results for the Juliet Test Suite align with findings from previous studies, we can confidently assert the validity of our methods, which in turn strengthens the reliability of our results on the PrimeVul dataset.

5.2.2 Source criticism

Throughout this work, we have primarily relied on peer-reviewed sources to ensure the credibility and academic integrity of the research. Initially an exception was made for the source for the PrimeVul dataset which was a preprint since it was the largest dataset based on realistic data. However it was peer-reviewed and presented in a conference in May 2025, prior to this study.

5.2.3 Threats to validity

Several factors may impact the validity of this study’s findings. The following subsection outlines specific threats to validity identified during the research process and discusses the steps taken to mitigate their impact.

lack of manual verification

The first threat is the lack of manual verification of the output of the SATs. If a tool reports any vulnerability within the marked function, it will count. The automation process does not check what kind of vulnerability was reported; it only checks if it was reported at all. Therefore, a tool might report the wrong vulnerability in the correct function, and it will still count as a TP. Some manual verification was performed, as explained in Section 5.2.3, but a higher level of manual verification for both Juliet and PrimeVul would take too much time for the span of this thesis. Based on a preliminary review of the results, this appears to have had a minimal impact on the overall findings.

Different tool outputs

The second issue arises in how different tools produce warnings. For example, if there is a memory leak issue, some tools would report it where the memory was created and some at the end of the function when the memory was not freed. This was solved by annotating that the vulnerable area is the entire function. This was done in part for this reason and partly

because the ground truth given for PrimeVul was on the function level, so it was reasonable to have the same level across both of our datasets. This could lead to the first threat to validity to increase.

Additional Vulnerabilities

Another threat to the validity of both datasets is that there is at least one confirmed vulnerability; it is tough to guarantee that the source code does not contain additional, undetected security flaws. This risk grows larger for the PrimeVul dataset that is based on realistic historical code where a piece of code can be deemed benign for a long time and later found vulnerable, such as the heartbleed vulnerability⁴. There could also be vulnerabilities that are undetected in the benign labeled data point. We minimize this risk by keeping the level we are analyzing as small as possible on a function level instead of a file level and minimizing the area where rogue vulnerabilities could be found, as well as using freshly labeled ground truth with PrimeVul, which was created in 2024.

Dataset and tool Limitations

A standard limitation affecting all tools is that not every project or commit in the PrimeVul dataset could be cloned, resulting in fewer total data points available for analysis. Additionally, some tools had limitations as described in Chapter 3.2.2, for example, Clang-Tidy requiring project compilation, explained in Section 3.2.3. Semgrep is struggling with large projects and limited executions on the Snyk License, causing variation in the subsets each tool analyzed, as seen in Table 4.6. These differences could potentially impact the comparative evaluation of tool performance. Table 5.3 summarizes the analysis method and main limitations for each tool and indicates how many data points in PrimeVul remained after exclusions. Some of the data points could not be built and analyzed, resulting in a DNF. This could be due to various reasons, such as the tool needing a compiled database, but it could not be provided (Clang-Tidy), running out of runs (Snyk), the commit URL not being fetched, or the tool crashing.

Table 5.3: Tool-specific limitations and remaining datapoints from real-world dataset.

Tool	Analysis method	Limitation	Datapoints
Clang-Tidy	AST-based	Projects require compilation	2260
Semgrep	Rule-based	Project size too large	4210
Snyk	AI based Tool	Limited usage	1570
Cppcheck	Unsound flow sensitive analysis	-	4094
Flawfinder	Pattern Matching	-	3978

This threat to validity is mitigated by the overall large size of the dataset, ensuring enough data remains for robust analysis despite these exclusions. Furthermore, much overlap among tool subsets still exists, as common data points often excluded (such as those from the Linux or Chromium projects due to their extensive size) were consistent across tools.

Tool configuration

The tools were run using the flags detailed in Table 3.2, and due to computational constraints, they were run on separate machines. It is possible that adjusting or optimizing these flags could give improved results. Additionally, some tools provide functionality to incorporate custom rules, which could significantly enhance vulnerability detection coverage.

⁴<https://heartbleed.com/>

Verification

To ensure accurate classification for both synthetic and real-world data, spot-checks were done by manually verifying script outputs. This involved looking at raw tool output files for selected data points and comparing them against their corresponding entries in the Prime-Vul dataset or directly within Juliet Test Suite files. For TP and FP, manual classifications were confirmed by checking whether the flagged issues matched the recorded vulnerabilities. Similarly, TN and FN were verified by confirming that the tools correctly did not flag vulnerabilities for specific data points.

Although sample-based verification cannot guarantee correctness since no errors were found in the testing process, they provide confidence in the accuracy and reliability of the classification scripts.

5.3 The work in a wider context

As software security grows increasingly critical in our rapidly digitalizing world, understanding the limitations and capabilities of security tools plays a crucial role. Gathering information about how SATs perform on realistic data could inform developers using SATs of the limitations, and the following areas are mitigated.

5.3.1 Security fatigue

Security fatigue occurs when maintaining security becomes too complex or burdensome for users, even when protocols are in place [14]. In the case of SATs, this could mean that running multiple tools and addressing every reported issue becomes overwhelming for developers. This highlights the importance of understanding the accuracy and reliability of these tools. A situation may arise where a developer must sift through nine FPs to identify and fix one TP, leading them to question whether the effort is worthwhile due to the fatigue it causes. However, in industries where high-security standards are critical, every flagged issue must be carefully reviewed to ensure that no TP is overlooked.

5.3.2 Automation in security

One of the motivations behind this study is that approximately 50% of open-source projects use SATs [1]. However, as the results demonstrate, many vulnerabilities remain undetected. The risk lies in developers assuming that no vulnerabilities exist simply because SATs do not flag any issues, creating a false sense of security. Over-reliance on automated tools without manual verification can be dangerous, as it may leave critical security flaws unidentified.

5.3.3 lack of tool transparency

A major challenge in evaluating SATs is the lack of transparency regarding how these tools detect vulnerabilities. This is particularly true for AI-driven SATs, as AI models are inherently black-box systems, making it difficult to understand their decision-making process. Additionally, subscription-based SATs often provide only broad, high-level descriptions of their internal workings without offering detailed insights into their detection mechanisms.

In contrast, open-source tools are not designed for profit and do not actively advertise their technologies. While some information about these tools can be found in their documentation, it is often written primarily as a byproduct of explaining how to use the tool rather than as a description of how the tool actually works. This lack of clarity can make it challenging for developers to fully trust or optimize the tools for their specific security needs.



6 Conclusion

The primary purpose of this thesis has been to evaluate SATs based on different technologies when applied to synthetic or realistic data. The study shows that SATs detect only a small fraction of C/C++ vulnerabilities, especially in real-world code. The tools evaluated were Clang-Tidy, Semgrep, Snyk, Cppcheck, and Flawfinder on the synthetic dataset Juliet and the realistic dataset PrimeVul. This chapter answers the research questions and outlines future work.

6.1 Research questions

The following section will present answers to the questions posed in Section 1.3, based on the results in Chapter 4 and the discussion in Chapter 5.

6.1.1 RQ1.1: How do static analysis tools based on different technologies perform when applied to synthetic datasets?

When applying SATs to synthetic datasets, most tools demonstrated limited predictive capabilities. The highest F1-score observed was 20.5%, achieved by Clang-Tidy, while the highest Matthews Correlation Coefficient (MCC) was 0.14, recorded by Cppcheck. These figures suggest that while certain tools can identify specific vulnerable patterns present in the synthetic dataset, their overall effectiveness is still limited.

Combining multiple tools can improve precision at the cost of a total number of correctly identified vulnerabilities decreasing. For instance, the intersection of Snyk, Semgrep, and Cppcheck achieved better ratio between TP and FP, indicating that agreement across tools may point to more reliable findings. This improvement is by no means consistent across different tools, as combining Snyk, Clang-Tidy and Flawfinder results in more FP than TP.

6.1.2 RQ1.2: How do static analysis tools based on different technologies perform when applied to real-world projects?

The performance of SATs on real-world data was significantly worse than on synthetic datasets. The highest F1-score, 8.4%, was again achieved by Clang-Tidy, while the top MCC was just 0.007, reported by Semgrep. This indicated a performance comparable to random guessing. The tool that flagged the most functions was Flawfinder, with 103 marked data points out of 4 764 in the dataset. However, the overall detection rates are so low that any fine-grained analysis becomes difficult to justify. These results highlight a limitation in current SATs when applied to complex real-world source code and underline the need for improvement in both tool capability and evaluation methods.

6.1.3 Updated Dataset

The realistic component of this study is based on the pairwise-labeled portion of the PrimeVul dataset. During the preparation phase, several issues were identified within the original dataset, prompting preprocessing and cleanup steps as outlined in Section 3.2.1. These included removing unreachable projects and ensuring accurate line-level mapping.

To support future research and reproducibility, a cleaned and preprocessed version of the dataset is made publicly available¹.

6.2 Future work

While this thesis provides a comprehensive evaluation of SATs on both synthetic and real-world datasets, it also raises important questions and identifies several areas for further research. These will be ordered below from most important to least important.

6.2.1 What is the best method to evaluate and compare SATs?

Every dataset used for evaluating SATs presents some form of limitation. Synthetic datasets, such as Juliet, are often less complex in terms of data flow and contain bugs that are overly distinct or simplified compared to those found in real-world code.

The realistic dataset used in PrimeVul suffers from the fact that it originates from open-source projects, where automated SATs may already have been applied before the code is merged. This means that the code in the main branches may have already been filtered by such tools, potentially reducing the number of detectable vulnerabilities. If a vulnerability is old and easily identifiable through known rule sets, modern tools might already have incorporated specific detections and skewing evaluation.

Another possibility is using a method where vulnerabilities are injected into realistic code [9]. This suffers from much of the same problems as synthetic data, such as simplistic and distinct bugs, but it fixes the issues of simplistic and unrealistic dataflow.

Currently, a good dataset combines realistic, synthetic, and injected vulnerabilities in a way that enables a holistic and unbiased evaluation of SATs. Such a dataset would overcome the weaknesses of both synthetic and historical sources. However, identifying or creating such a dataset remains a non-trivial challenge and an important direction for future work.

¹https://drive.google.com/file/d/1t6ovT_fODMiMl_fXP4yJwO500I0BRwkD/view?usp=drive_link

6.2.2 Creating a new tool

Given the bad results of these tools another possibility for future work is the development of a new SATs tailored specifically for vulnerability detection in modern software systems. A new solution could focus on addressing the limitations identified in both rule-based and learning-based tools, potentially combining their strengths in a hybrid approach. Emphasis could also be placed on improving usability, configurability, and transparency, particularly in tools designed for open-source or resource-constrained projects.

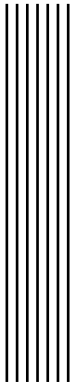
6.2.3 More languages and more Tools

A natural direction for future work involves expanding the scope of static analysis evaluations to include a broader range of programming languages and analysis tools. This study focused exclusively on C and C++ code, primarily due to the structure of the datasets and the availability of well-established tools targeting these languages. Software systems are written in more languages than C/C++, such as Python, Java, Rust, or Go, each with unique security characteristics and common vulnerability types. Evaluating SATs across multiple languages would provide a more comprehensive understanding of their generalizability and practical applicability in diverse development environments.

Moreover, this study only examined a subset of available tools. Looking into other commercial solutions and newer machine learning-based models may offer improved detection capabilities and adaptability. Furthermore, investigating how the tools are integrated into modern CI/CD pipelines would be interesting since a lot of projects use some sort of automated SATs. Such a study could likely use a similar approach to the one in this paper.

Another area to explore would be expanding the study beyond static analysis to include methods such as dynamic analysis. Dynamic analysis can provide real-time feedback and detect issues that static analysis might miss. However, this approach would require the projects to be built and executed as part of the analysis process.

Finally, further research could explore hybrid approaches that combine static and dynamic analysis or integrate multiple tools into a unified pipeline. Understanding how these multi-tool or multi-technique strategies perform compared to standalone tools could significantly benefit both researchers and practitioners aiming to optimize their security workflows.



Bibliography

- [1] Moritz Beller, Radjino Bholanath, Shane McIntosh, and Andy Zaidman. “Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software”. In: *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. Vol. 1. 2016, pp. 470–481. DOI: 10.1109/SANER.2016.105.
- [2] Paul E. Black. *Juliet 1.3 Test Suite: Changes From 1.2*. Tech. rep. National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.TN.1995.
- [3] Wachiraphan Charoenwet, Patanamon Thongtanunam, Van-Thuan Pham, and Christoph Treude. “An Empirical Study of Static Analysis Tools for Secure Code Review”. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. Vienna, Austria: Association for Computing Machinery, 2024, pp. 691–703. ISBN: 9798400706127. DOI: 10.1145/3650212.3680313.
- [4] Brian Chess and Gary McGraw. “Static Analysis for Security”. In: *IEEE Secur. Priv.* 2 (2004), pp. 76–79. URL: <https://api.semanticscholar.org/CorpusID:262254294>.
- [5] Davide Chicco. “Ten quick tips for machine learning in computational biology”. In: *BioData Mining* 10.1 (Dec. 8, 2017), p. 35. ISSN: 1756-0381. DOI: 10.1186/s13040-017-0155-3. (Visited on 04/01/2025).
- [6] Terry Cohen, Damien Cupif, Aurelien Delaitre, Charles, Elizabeth Fong, and Vadim Okun. “Improving Software Assurance through Static Analysis Tool Expositions”. en. In: 5 (2017-10-31 2017).
- [7] José D’Abruzzo Pereira and Marco Vieira. “On the Use of Open-Source C/C++ Static Analysis Tools in Large Projects”. In: *2020 16th European Dependable Computing Conference (EDCC)*. 2020, pp. 97–102. DOI: 10.1109/EDCC51268.2020.00025.
- [8] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. “Vulnerability Detection with Code Language Models: How Far Are We?” In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, 2025, pp. 469–481. URL: <https://www.computer.org/csdl/proceedings-article/icse/2025/056900a469/215aWRJLUzy>.

- [9] Brendan Dolan-Gavitt, Patrick Hulin, Engin Kirda, Tim Leek, Andrea Mambretti, Wil Robertson, Frederick Ulrich, and Ryan Whelan. "LAVA: Large-Scale Automated Vulnerability Addition". In: *2016 IEEE Symposium on Security and Privacy (SP)*. 2016, pp. 110–121. DOI: 10.1109/SP.2016.15.
- [10] Matteo Esposito, Valentina Falaschi, and Davide Falessi. "An Extensive Comparison of Static Application Security Testing Tools". In: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. EASE '24. Salerno, Italy: Association for Computing Machinery, 2024, pp. 69–78. ISBN: 9798400717017. DOI: 10.1145/3661167.3661199.
- [11] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries". In: *Proceedings of the 17th International Conference on Mining Software Repositories*. MSR '20. Seoul, Republic of Korea: Association for Computing Machinery, 2020. ISBN: 9781450375177. DOI: 10.1145/3379597.3387501.
- [12] Anum Fatima, Shazia Bibi, and Rida Hanif. "Comparative study on static code analysis tools for C/C++". In: *2018 15th International Bhurban Conference on Applied Sciences and Technology (IBCAST)*. 2018, pp. 465–469. DOI: 10.1109/IBCAST.2018.8312265.
- [13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. 2020. arXiv: 2002.08155 [cs.CL].
- [14] Steven Furnell and Kerry-Lynn Thomson. "Recognising and addressing 'security fatigue'". In: *Computer Fraud Security* 2009.11 (2009), pp. 7–11. ISSN: 1361-3723. DOI: 10.1016/S1361-3723(09)70139-3.
- [15] Christoph Gentsch. "Evaluation of Open Source Static Analysis Security Testing (SAST) Tools for C". In: (Jan. 2020).
- [16] "IEEE Standard Glossary of Software Engineering Terminology". In: *IEEE Std 610.12-1990* (1990), pp. 1–84. DOI: 10.1109/IEEESTD.1990.101064.
- [17] Federal Bureau of Investigation. *2024 Internet Crime Report*. Tech. rep. FBI IC3, 2024. URL: https://www.ic3.gov/AnnualReport/Reports/2024_IC3Report.pdf.
- [18] Arvinder Kaur and Ruchikaa Nayyar. "A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code". In: *Procedia Computer Science* 171 (2020), pp. 2023–2029. ISSN: 1877-0509. DOI: 10.1016/j.procs.2020.04.217.
- [19] James C. King. "Symbolic execution and program testing". In: *Commun. ACM* 19.7 (July 1976), pp. 385–394. ISSN: 0001-0782. DOI: 10.1145/360248.360252.
- [20] Valentina Lenarduzzi, Francesco Pecorelli, Nyyti Saarimaki, Sebastian Lujan, and Fabio Palomba. "A critical comparison on six static analysis tools: Detection, agreement, and precision". In: *Journal of Systems and Software* 198 (2023), p. 111575. DOI: 10.1016/j.jss.2022.111575.
- [21] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. "SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities". In: *IEEE Transactions on Dependable and Secure Computing* 19.04 (July 2022), pp. 2244–2258. ISSN: 1941-0018. DOI: 10.1109/TDSC.2021.3051525.
- [22] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection". In: *Proceedings 2018 Network and Distributed System Security Symposium*. NDSS 2018. Internet Society, 2018. DOI: 10.14722/ndss.2018.23158.

-
- [23] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. “An empirical study on the effectiveness of static C code analyzers for vulnerability detection”. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2022. Virtual, South Korea: Association for Computing Machinery, 2022, pp. 544–555. ISBN: 9781450393799. DOI: 10.1145/3533767.3534380.
 - [24] Fabio Massacci and Viet Hung Nguyen. “Which is the right source for vulnerability studies? an empirical analysis on Mozilla Firefox”. In: *Proceedings of the 6th International Workshop on Security Measurements and Metrics*. MetriSec ’10. Bolzano, Italy: Association for Computing Machinery, 2010. ISBN: 9781450303408. DOI: 10.1145/1853919.1853925.
 - [25] Radhia Cousot Patrick Cousot. “Abstract Interpretation Framework”. In: *Journal of Logic and Computation*, 1992. DOI: 10.1093/logcom/2.4.511.
 - [26] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. arXiv: 1910.10683 [cs.LG].
 - [27] Alaa Tharwat. “Classification Assessment Methods: a detailed tutorial”. In: (Aug. 2018). DOI: 10.1016/j.aci.2018.08.003.
 - [28] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. *CodeT5+: Open Code Large Language Models for Code Understanding and Generation*. 2023. arXiv: 2305.07922 [cs.CL].
 - [29] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. *CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation*. 2021. arXiv: 2109.00859 [cs.CL].
 - [30] Qiuming Zhu. “On the performance of Matthews correlation coefficient (MCC) for imbalanced dataset”. In: *Pattern Recognition Letters* 136 (2020), pp. 71–80. ISSN: 0167-8655. DOI: 10.1016/j.patrec.2020.03.030.

A

Large-Language Models

AI and large language models (LLMs) are becoming increasingly popular in software development due to their effectiveness and ability to perform a wide range of tasks. In this study, we evaluate two transformer-based LLMs, CodeBERT and T5. These models are fine-tuned for binary classification to detect vulnerabilities in source code. They were selected for their popularity, demonstrated effectiveness in recent research, and relatively low computational cost for fine-tuning compared to larger models [8, 28].

CodeBERT

CodeBERT is a bimodal transformer trained on paired natural language and code across six programming languages [13]. Its pretraining objectives enable it to capture both syntax and semantic relationships in code, making it a strong candidate for vulnerability detection.

T5 / CodeT5

T5 is a sequence-to-sequence transformer that treats every task as text generation [26]. CodeT5 adapts this model for source code, pre-trained on large GitHub repositories [29]. This allows it to better model code semantics and perform well on various software engineering tasks, such as vulnerability detection by code classification.

Dataset

Pretrained models were retrieved from the HuggingFace model repository ^{1,2}. The PrimeVul dataset, conveniently partitioned into training, validation, and testing subsets, was used to fine-tune and evaluate the models ³.

¹<https://huggingface.co/microsoft/codebert-base>

²<https://huggingface.co/Salesforce/codet5p-220me>

³<https://github.com/DLVulDet/PrimeVul>

Training Procedure

Model training and computation were done using an NVIDIA A100 GPU hosted on Google Colab ⁴. Each function in the dataset was labeled as either vulnerable (1) or benign (0). We selected the training parameters to closely match those used in the study by Ding. et al. [8]. They are summarized in Table A.1. Because the PrimeVul Training set contains far more benign than vulnerable functions, we implemented class weighting of the rare class (vulnerabilities) to mitigate the imbalance.

Table A.1: Training parameters for LLM fine-tuning.

Parameter	Value
Epochs	4
Batch Size	128
Learning Rate	2e-5
Maximum Token Length	512
Rare Class Weight	10

Evaluation Methodology

After training and validation phases, the models were evaluated on the paired test set, a balanced dataset with equal amounts of benign and vulnerable functions. This is the same distribution as was used for evaluation of the SATs. Fundamental and advanced metrics were computed for both models. Additionally, the predicted labels, the associated function data, and ground truth labels were saved and used for later computation of pair-wise prediction metrics.

Accuracy

In addition to the metrics defined in the background chapter 2, accuracy was computed for a better comparison with PrimeVul Study [8]. Accuracy measures the overall effectiveness of the tool by evaluating how often it correctly classifies both vulnerable and non-vulnerable code. It considers all four classification outcomes, TP, TN, FP and TP. A high accuracy indicates that the tool makes correct predictions most of the time [27]. Accuracy is calculated as:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (A.1)$$

Results

Table A.2 summarizes the classification performance of CodeBERT and T5 on the PrimeVul.

Table A.2: Percentage distribution of classification results for CodeBERT and T5.

Tool	TP (%)	FP (%)	TN (%)	FN (%)
CodeBERT	10.0	10.2	39.8	40.0
T5	10.9	11.0	39.1	39.0

From the classification results, additional performance metrics were calculated based on the Equations 2.1, 2.2, 2.3 and 2.4. The results can be found in Table A.3.

⁴<https://colab.research.google.com/>

Table A.3: Precision, Recall, F1-score, and MCC for T5 and CodeBERT on the realistic dataset.

Tool	Precision (%)	Recall (%)	F1-score (%)	MCC (-1 – +1)
CodeBERT	49.5	20.0	28.5	-0.005
T5	49.8	21.8	30.4	-0.003

Comparison with Earlier Work

Table A.4 shows the values achieved in our evaluation along with the results from the study by Ding et al.[8]. The table contains Accuracy along with the Pair-Wise metrics in percentages, Correct Prediction (CP), Vulnerable Prediction (VP), Benign Prediction (BP), and Reversed Prediction (RP).

Table A.4: Performance compared with PrimeVul study.

Model	Acc. (%)	CP	VP	BP	RP
CodeBERT (PrimeVul)	96.7	1.06	12.9	84.8	1.24
CodeBERT (Ours)	49.8	1.1	18.8	78.4	1.6
T5 (PrimeVul)	96.9	1.77	11.3	86.2	0.71
T5 (Ours)	50.0	1.4	20.6	76.5	1.6

Discussion

Compared with the study by Ding et al.[8], we obtain much lower accuracy but pairwise scores as seen in Table A.4. A likely reason is that the PrimeVul authors evaluated on the full test set which is heavily imbalanced, with about 97% benign and only 3% vulnerable functions [8]. Its relatively simple to get high accuracy on such an imbalanced test-set, because classifying every function as benign would lead to 97% accuracy while not finding a single vulnerability.

Our study used the balanced PrimeVul test set with a 50/50 split for a fairer comparison against the SATs. Furthermore, the balanced test-set only contains functions that have over 80% string similarity between the vulnerable and benign versions, making the differences so small and subtle that they might be harder for LLMs to detect. Our models also flag slightly more vulnerabilities than the PrimeVul ones, possibly because of the different class weights.

Conclusion

During evaluation, The Matthews correlation coefficient remained close to zero, indicating performance close to random guessing. In practice, this means the models identify only one vulnerability in five and miss most flaws. The warnings they generate also lack the contextual details such as CWE labels, suspect lines, and remediation hints that SATs provide. Overall, these fine-tuned LLMs perform only on par with, and often worse than, conventional static analyzers while providing less information.

B Union Data

Table B.1: Union counts for TPs and FPs on the Juliet Dataset.

Combination	TP	FP
Singles		
Snyk	6074	3622
Semgrep	1445	835
Cppcheck	6687	3073
clang	15984	32182
Flawfinder	6877	11091
Pairs		
Snyk, Semgrep	6552	3900
Snyk, Cppcheck	12321	6548
Snyk, clang	20353	33877
Snyk, Flawfinder	12276	13827
Semgrep, Cppcheck	7767	3868
Semgrep, clang	17275	32879
Semgrep, Flawfinder	8190	11767
Cppcheck, clang	21295	34153
Cppcheck, Flawfinder	13158	13846
clang, Flawfinder	20266	38311
Triplets		
Snyk, Semgrep, Cppcheck	12715	6786
Snyk, Semgrep, clang	20721	34062
Snyk, Semgrep, Flawfinder	12680	14012
Snyk, Cppcheck, clang	25260	35750
Snyk, Cppcheck, Flawfinder	18153	16484
Snyk, clang, Flawfinder	24555	39954
Semgrep, Cppcheck, clang	22239	34810
Semgrep, Cppcheck, Flawfinder	14124	14482
Semgrep, clang, Flawfinder	21481	38913
Cppcheck, clang, Flawfinder	25393	40126
Quadruples		
Snyk, Semgrep, Cppcheck, clang	25562	35895
Snyk, Semgrep, Cppcheck, Flawfinder	18491	16629
Snyk, Semgrep, clang, Flawfinder	24885	40077
Snyk, Cppcheck, clang, Flawfinder	29278	41671
Semgrep, Cppcheck, clang, Flawfinder	26261	40688
All Five		
Snyk, Semgrep, Cppcheck, clang, Flawfinder	29542	41754

Table B.2: Union counts for TPs and FPs on the PrimeVul Dataset.

Combination	TP	FP
Singles		
Snyk	17	18
Clang-Tidy	52	50
Cppcheck	13	14
Flawfinder	52	51
Semgrep	12	10
Pairs		
Snyk, Clang-Tidy	56	54
Snyk, Cppcheck	27	29
Snyk, Flawfinder	58	58
Snyk, Semgrep	23	22
Clang-Tidy, Cppcheck	60	58
Clang-Tidy, Flawfinder	79	76
Clang-Tidy, Semgrep	57	53
Cppcheck, Flawfinder	58	57
Cppcheck, Semgrep	22	22
Flawfinder, Semgrep	55	53
Triplets		
Snyk, Clang-Tidy, Cppcheck	64	62
Snyk, Clang-Tidy, Flawfinder	82	80
Snyk, Clang-Tidy, Semgrep	60	56
Snyk, Cppcheck, Flawfinder	64	64
Snyk, Cppcheck, Semgrep	31	31
Snyk, Flawfinder, Semgrep	60	59
Clang-Tidy, Cppcheck, Flawfinder	83	80
Clang-Tidy, Cppcheck, Semgrep	63	59
Clang-Tidy, Flawfinder, Semgrep	82	78
Cppcheck, Flawfinder, Semgrep	60	58
Quadruplets		
Snyk, Clang-Tidy, Cppcheck, Flawfinder	86	84
Snyk, Clang-Tidy, Cppcheck, Semgrep	66	62
Snyk, Clang-Tidy, Flawfinder, Semgrep	84	81
Snyk, Cppcheck, Flawfinder, Semgrep	65	64
Clang-Tidy, Cppcheck, Flawfinder, Semgrep	85	81
All Five		
Snyk, Clang-Tidy, Cppcheck, Flawfinder, Semgrep	87	84